

stable kernel testing using perf

6:27 pm

Testing stable Linux kernels with perf helps ensure new stable updates do not introduce performance regressions, high latencies, or CPU bottlenecks. The perf subsystem provides direct access to hardware performance counters and kernel tracepoints, allowing you to run micro-benchmarks, trace system calls, and execute internal self-tests. [\[1, 2, 3, 4, 5\]](#)

The primary workflows for testing stable kernels using perf are detailed below.

1. Execute perf Internal Subsystem Self-Tests

The perf tool contains a built-in suite designed to stress and validate the stability of its own kernel infrastructure (perf_events). This ensures the patch releases haven't broken the kernel-to-user ABI. [\[1, 2, 3, 4, 5\]](#)

- **Run all tests:** Automatically execute all functional and structural checks.

bash

```
sudo perf test
```

Use code with caution.

- **Run specific test suites:** Target specific kernel subsystems like the BPF skeleton or thread maps.

bash

```
sudo perf test BPF
sudo perf test "Track kmem alloc"
```

Use code with caution.

[\[1\]](#)

2. Compare CPU Cycles and Overhead (perf stat) [\[1, 2\]](#)

When testing a new stable kernel against an older release, perf stat provides a structural overview of hardware efficiency metrics. [\[1, 2\]](#)

- **Run a benchmark matrix:** Track core architectural metrics under a heavy testing payload.

bash

```
sudo perf stat -B dd if=/dev/zero of=/dev/null count=1000000
```

Use code with caution.

- **Verify architectural events:** Isolate performance indicators like CPU cycles, page faults, and context switches.

bash

```
sudo perf stat -e cpu-cycles,page-faults,context-switches -a sleep 10
```

Use code with caution.

Compare the output data between kernel versions. A sudden spike in page faults or context switches suggests a kernel regression. [\[1, 2, 3, 4, 5\]](#)

3. Record and Profile Kernel Latency (perf record & perf report) [\[1\]](#)

To diagnose if a stable patch introduces CPU locking or slow function execution, profile the kernel under a realistic system load. [\[1, 2\]](#)

- **Sample system-wide kernel activity:**

bash

```
sudo perf record -F 99 -a -g -- sleep 15
```

Use code with caution.

- `-F 99`: Samples 99 times per second to prevent high overhead.
- `-a`: Captures system-wide data across all CPUs.
- `-g`: Captures call graphs to trace function parentage.

- **Analyze the profile:**

bash

```
sudo perf report --stdio
```

Use code with caution.

Look for functions marked with `[k]` (indicating the kernel space). If specific primitives like `_raw_spin_lock` dominate the top of the report, it implies lock contention issues in the new patch. [\[1, 2, 3, 4\]](#)

4. Trace System Call Latencies (`perf trace`) [\[1\]](#)

Stable kernel updates often patch file systems or memory management, making it essential to audit system call execution overhead. [\[1, 2, 3\]](#)

- **Track global system call performance:** Evaluate system call latency with lower resource overhead than standard tracing utilities.

bash

```
sudo perf trace --summary sleep 10
```

Use code with caution.

- **Identify blocking behavior:** Isolate system calls that block execution loops to capture real-time application stalls.

bash

```
sudo perf trace -e openat,read,write --duration 10.0
```

Use code with caution.

[\[1, 2\]](#)

Best Practices for Stable Kernel Regression Analysis

- **Match tool and kernel versions:** Always match your `perf` userspace binary variant directly with the compiled source code version of the target stable kernel to ensure complete trace compatibility.

- **Maintain identical system configurations:** Ensure boot parameters, active daemon services, and power management features (`cpufreq` governors) are locked identically between kernel tests.
- **Run iterative baselines:** Isolate performance differences by executing each test payload multiple times across both kernels, discarding anomalies to ensure changes represent true kernel variations rather than hardware jitter. [[1](#), [2](#), [3](#)]